Laboratorul 1

Introducere

Memoria – chestionar și observații

Unitatea	Număr de bytes	Aproximare
kilobyte	2^{10} (1024) bytes	10^3 bytes
megabyte	2^{20} (1 048 576) bytes	10^6 bytes
gigabyte	2^{30} (1 073 741 824) bytes	10^9 bytes
terabyte	2^{40} bytes	10^{12} bytes

Operațiile cu Memoria

- Calculatoare de uz general folosesc un set de instrucțiuni numit program pentru a procesa datele.
- Un computer execută programul pentru a crea date de ieșire pornind de la datele de intrare
- Instrucțiunile programului și operanzii de date sunt stocate în memorie
- Două operații sunt de bază şi necesită acces la memorie:

  1 - operațiile de încărcare (citire sau fetch) unde conținutul locației de memorie specificate este citit de către procesor

  2 - operaţiile de stocare (scriere) - data de la procesor este stocată în locația de memorie specificată

Operațiile cu Memoria

Arhitectura setului de instrucțiuni: setul de instrucțiuni ale procesorului

Sunt 4 tipuri de operaţii de bază :

- Transferul de date între memorie și un registru procesor.
- Operarţiile aritmetice și logice.
- Secvența program şi secvența de control.
- Transfer I/O.

Operațiile cu Memoria

Ciclul de instrucțiuni de bază

- Ciclul de bază de operare calculator

  - Extragerea (FETCH) instrucțiunii din memorie într-un registru de control (PC)
  - Decodarea instrucțiunii
  - Localizarea operanzilor utilizati de instrucțiune
  - Aducerea operanzilor din memorie (dacă este necesar)
  - Execută opratia in regiștrile procesor
  - Stocheaza resultatul in locatia potrivita
  - Reia ciclul de la pasul 1 pentru a prelua următoarea instrucțiune

Operațiile cu Memoria

EXECUȚIA INSTRUCȚIUNII ȘI SECVENȚIEREA LINIARĂ

```
Addresă        Conţinut          Observații

   i           Mov R0,[A]        Trei instrucţiuni
 i+4           Add R0,[B]        ale
 i+8           Mov [C],R0        programului
               .
               .
               .
              A                 <---
               .                   |
               .                   |
               .                   |   Data ptr. program
              B                 <---     C <- [A]+[B]
               .                   |
               .                   |
              C                 <---
```

Operațiile cu Memoria

PC - Numărătorul de Program: ține adresa instrucțiunii următoare ce urmează să fie executată.

• Secvențierea liniară (directă) : dacă aducerea și executarea instrucțiunilor se efectuează una câte una, din adresele succesive de memorie, aceasta se numește secvențierea liniară.

• Sunt două faze importante de execuție a instrucțiunilor:

- faza de aducere din memorie a instrucțiunii: instrucțiunea este preluată din memorie și este plasată în registru de instrucțiuni IR.
- faza de execuție a instrucțiunii: conținutul IR este decodificat și procesorul efectuează operația fie prin citirea datelor din memorie fie din regiștrii.

Memoria – chestionar și observații

Întrebarea 1

Un PC are 32 MB memorie. De câţi biţi este nevoie pentru a adresa un singur byte de memorie?

Întrebarea 2

Un PC are 128 MB memorie. Fiecare cuvânt al acestui computer are 8 bytes.De câţi biţi este nevoie pentru a adresa un singur cuvânt de memorie?

Tipuri de adresare pe byte

Little Endian (e.g., DEC, Intel):

» byte-ul de ordin mai mic stocat la adresa cea mai mică

» byte0 byte1 byte2 byte3

- Eg: 46,78,96,54 (32 bit data)
- H BYTE L BYTE

Big Endian

- Big Endian (e.g. IBM, Motorolla, Sun, HP)

  » byte-ul de ordin înalt stocat la adresa cea mai mică

  » byte3 byte2 byte1 byte0

- Programatorii trebuie sa fie atenţi când transferă datele între o maşină Big Endian şi una Little Endian

Memoria – chestionar și observații

Întrebarea 1

Un PC are 32 MB memorie. De câţi biţi este nevoie pentru a adresa un singur byte de memorie?

- $R1=2^{20}+2^{5}=2^{25}$ -> 25 biți ;

Întrebarea 2

Un PC are 128 MB memorie. Fiecare cuvânt al acestui computer are 8 bytes.De câţi biţi este nevoie pentru a adresa un singur cuvânt de memorie?

- $R2=(2^{20}+2^{7})- 2^{3}=2^{24}$-> 24 biți

Memoria – chestionar și observații

Alinierea memoriei

Magistrala de date stabileşte şi tipul de aliniere a memoriei:

- Cazul 16 biţi de date : cuvintele aliniate încep la adresele 0,2,4,6, ...
- Cazul 32 biţi de date : cuvintele aliniate încep la adresele 0,4,8,12, ...
- Cazul 64 biţi de date : cuvintele aliniate încep la adresele 0,8,16,32, ...
- Memorie nealiniată (adrese arbitrare ale bytes)

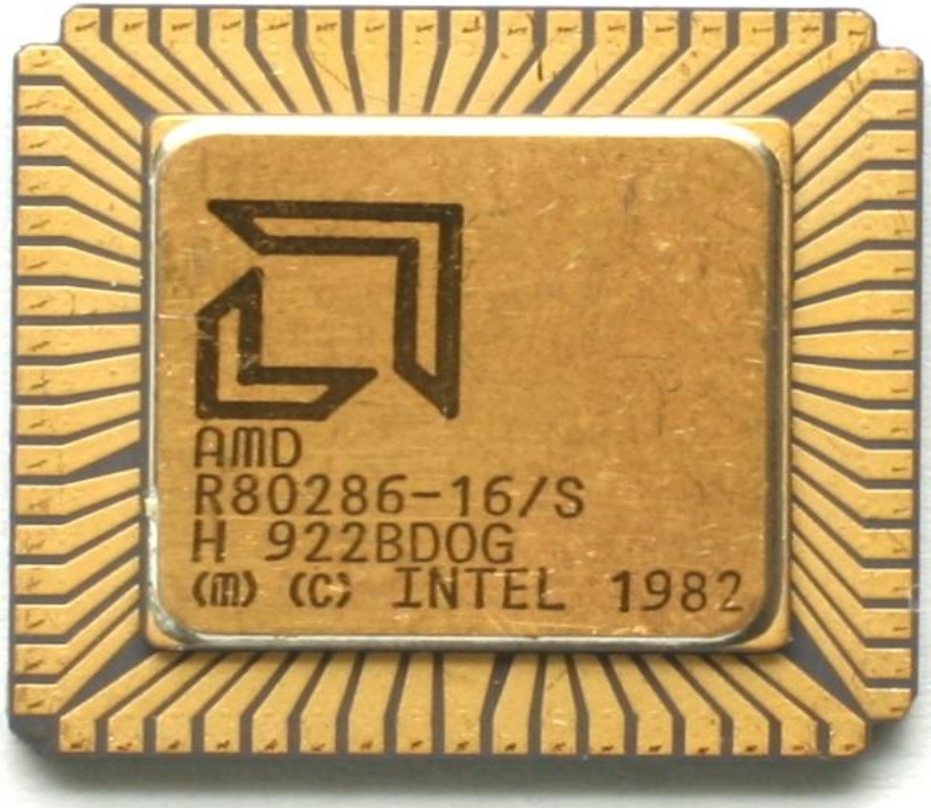
Operațiunile cu memoria

- Calculatoare de uz general folosesc un set de instrucțiuni numit program pentru a procesa datele.
- Un computer execută programul pentru a crea date de ieșire pornind de la datele de intrare
- Instrucțiunile programului și operanzii de date sunt stocate în memorie
- Două operații sunt de bază şi necesită acces în memorie:
  1. Operațiune de încărcare (Citire sau Fetch) - onținutul locației de memorie specificate este citit de către procesor.
  2. Stocarea operaţiei (scriere) - data de la procesor este stocată în locația de memorie specificată

Microprocesoare și socket-uri

- Familia de microprocesoare iAPx86 stă la baza calculatoarelor IBM PC, începând de la procesoarele 8088/8086, continuând cu 80286, 80386, 80486, Pentium, etc. Procesorul 8086 reprezintă referința familiei ce este cunoscută pe scurt sub denumirea de familia microprocesoarelor x86. De aceea se vor face referiri în continuare la arhitectura 8086.

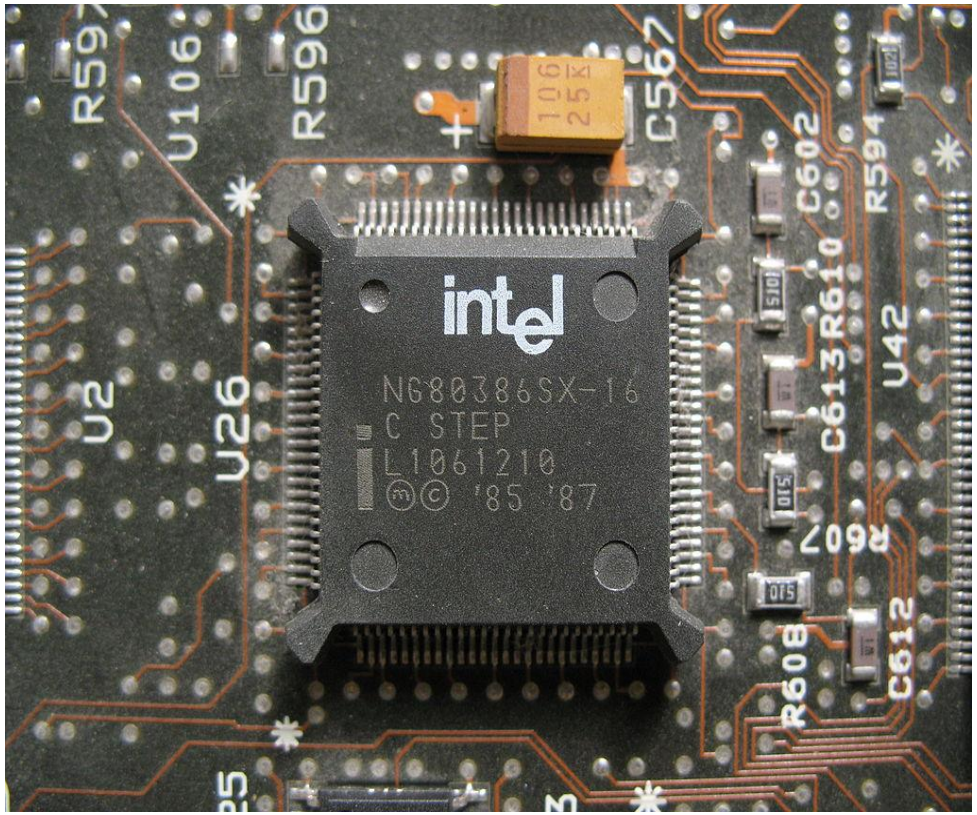
- Magistralele interne, regiștrii și memoria sunt organizate pe **16 biți** pentru *8088, 8086 și 80286*.



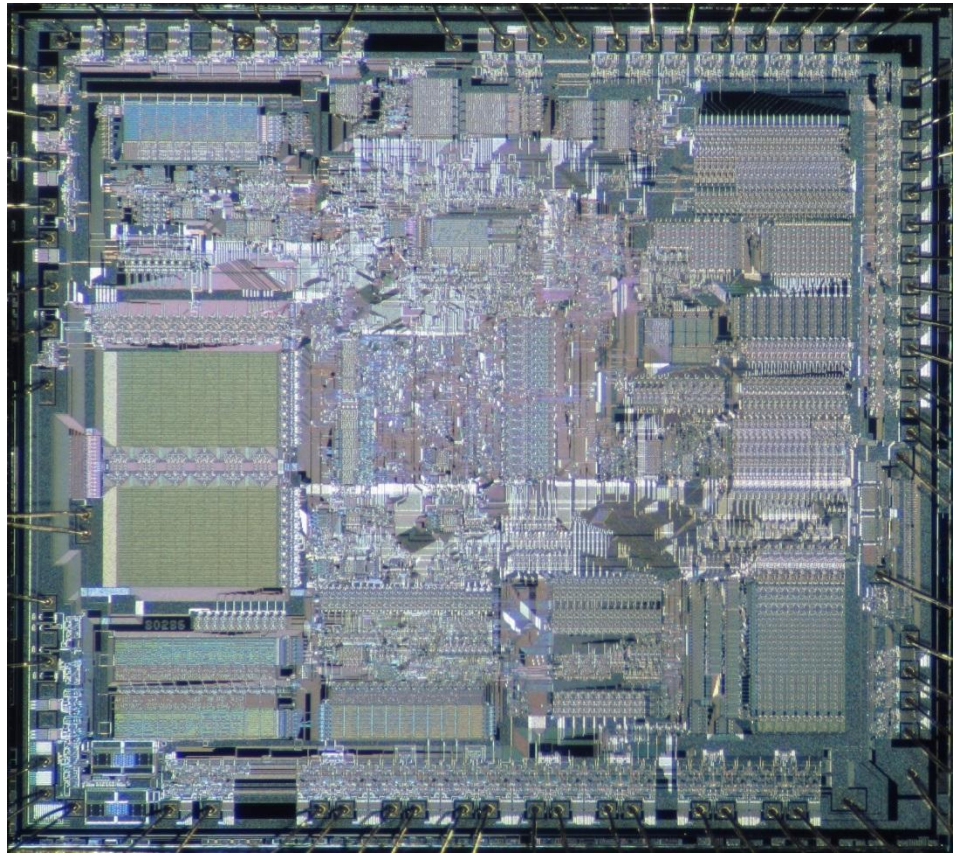
Microprocesorul AMD 80286 (16 biți date, 24 biți de adresa).

Microprocesoare și socket-uri

- Începând cu *80386* Intel a introdus magistralele pe **32 de biți**.



Precesorul Intel 80386SX pe o placa non-upgradabilă.



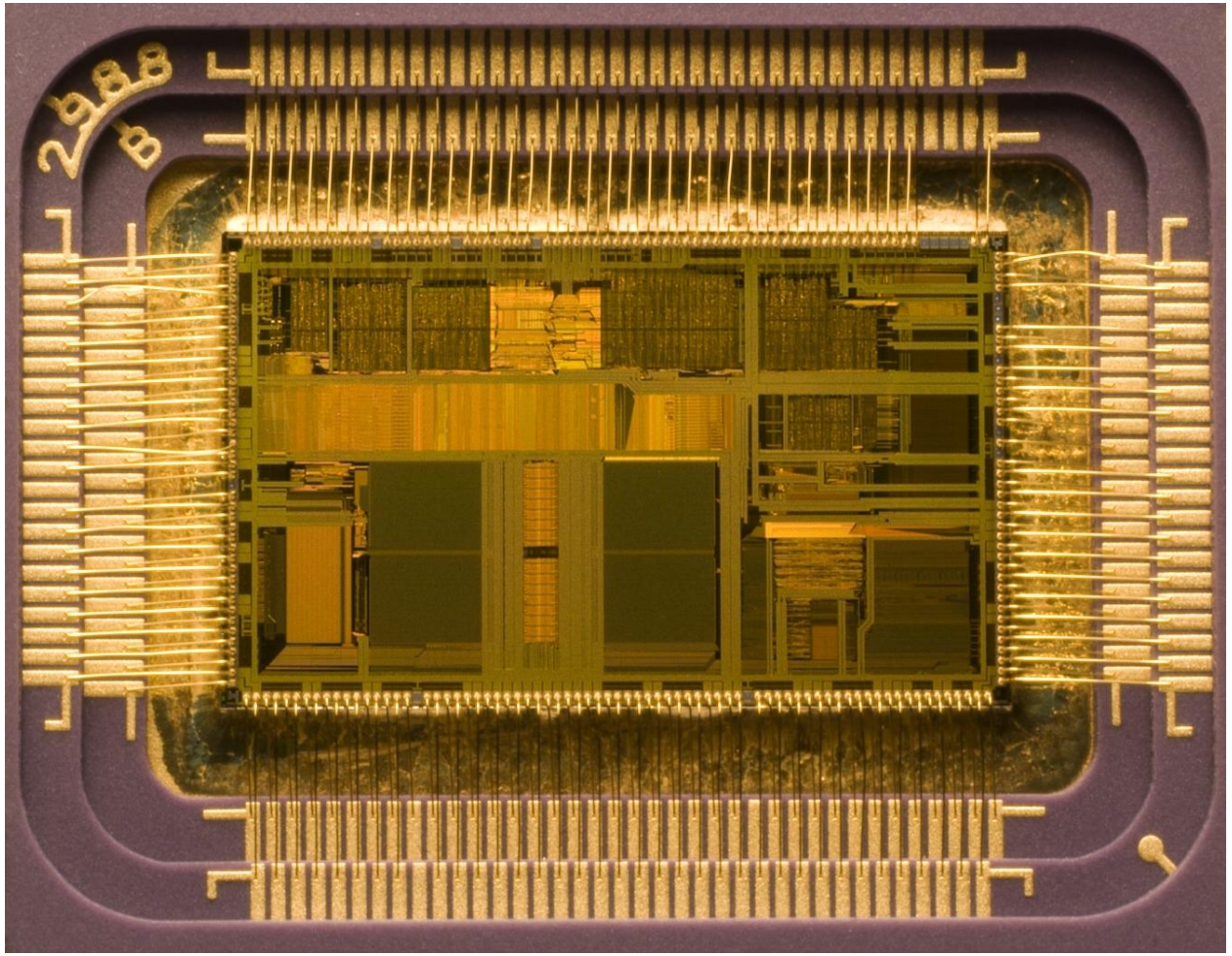
Microprocesorul AMD 80286 (structura fizică a CI).

Microprocesoare și socket-uri

Precesorul Intel 80386DX

Microprocesoare și socket-uri

Pentru 80486SLC (din 1992) s-au introdus 16Kb pentru memoria L1 cache dar adresarea memorie se face pe 16 biți.
Versiunea 80486DLC (1994) este utilată cu magistrale complete pe de 32 biți, ceea ce oferă un mare avantaj sistemului de operare.

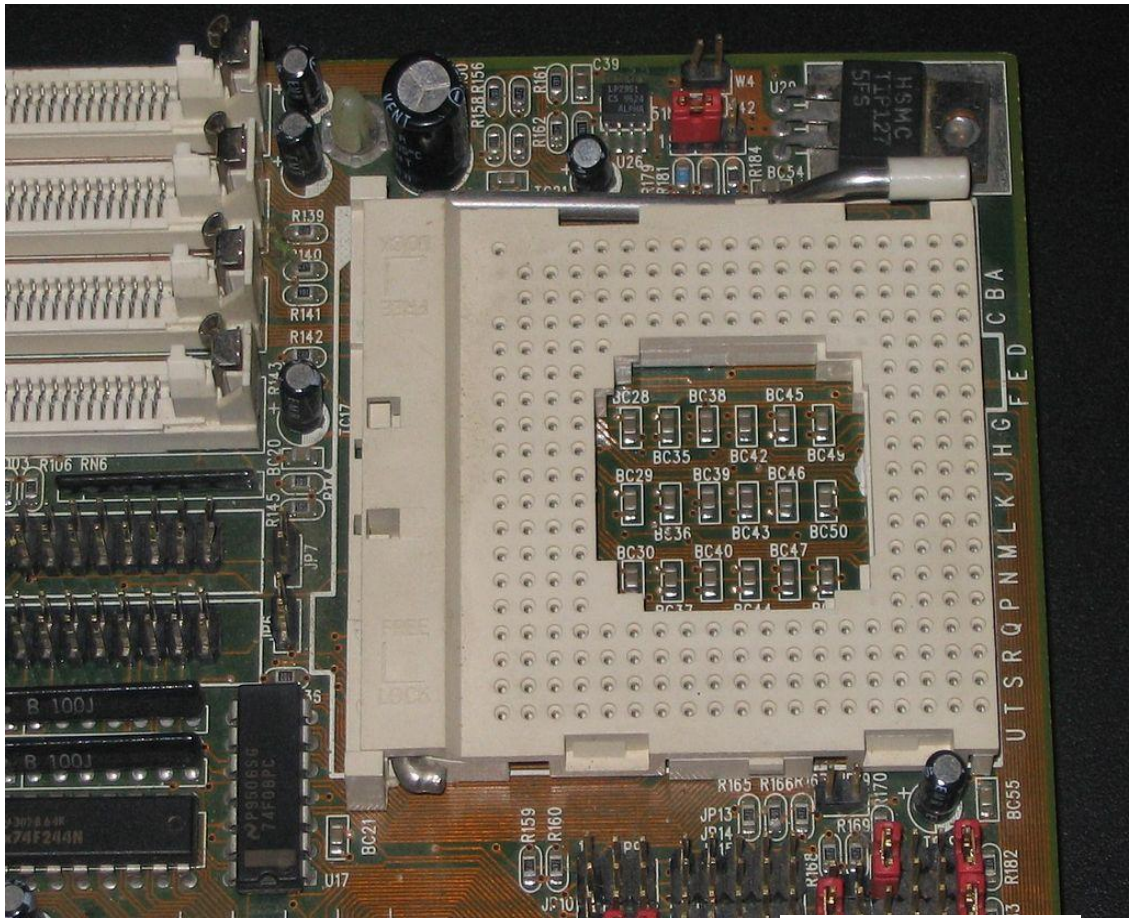


Microprocesorul Intel 80486DX2 – CI sectiune.
Magistralele de adrese și date pe 32 biți.

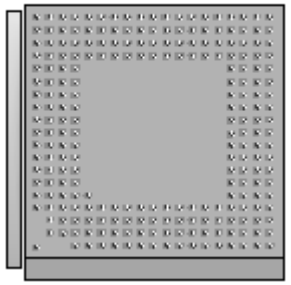
Microprocesoare și socket-uri

După socketul 3 au urmat:

- Socket 4 (273 pini – Intel Pentium, 66MHz),
- Socket 5 (320 pini, Intel Pentium , AMD K5, Cyrix 6x86),
- Socket 7 (321 pini, Intel Pentium MMX,...),
- Socket 8 (387 pini, Intel Pentium PRO),
- Slot 1 – 242 pini
- Slot 2 – 330 (Intel Pentium II/XENON).



Socket 3

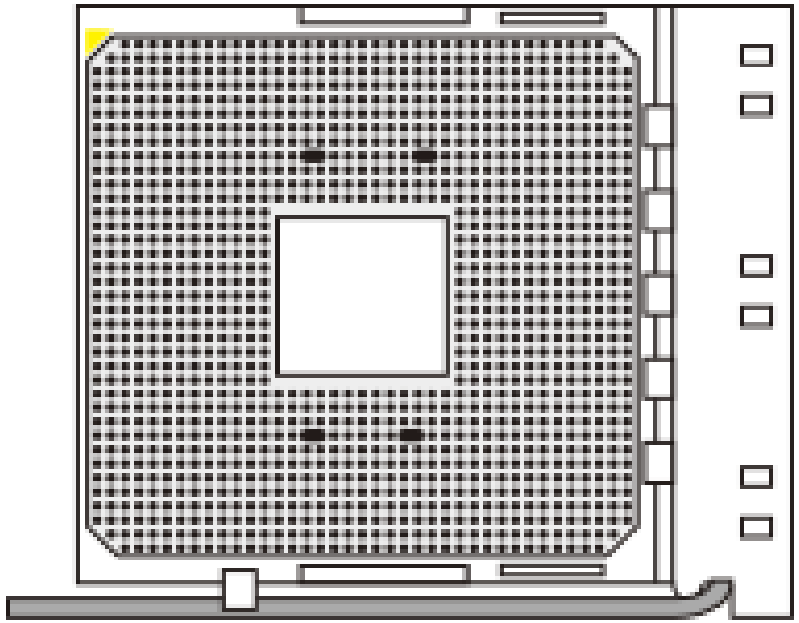


Socket 3 – Serie de socket-uri CPU cu 237 pini (Bus 486; 50 MHz), folosită pentru microprocesoarele din seria 80486 pâna la AMD Am5x86.

Microprocesoare și socket-uri

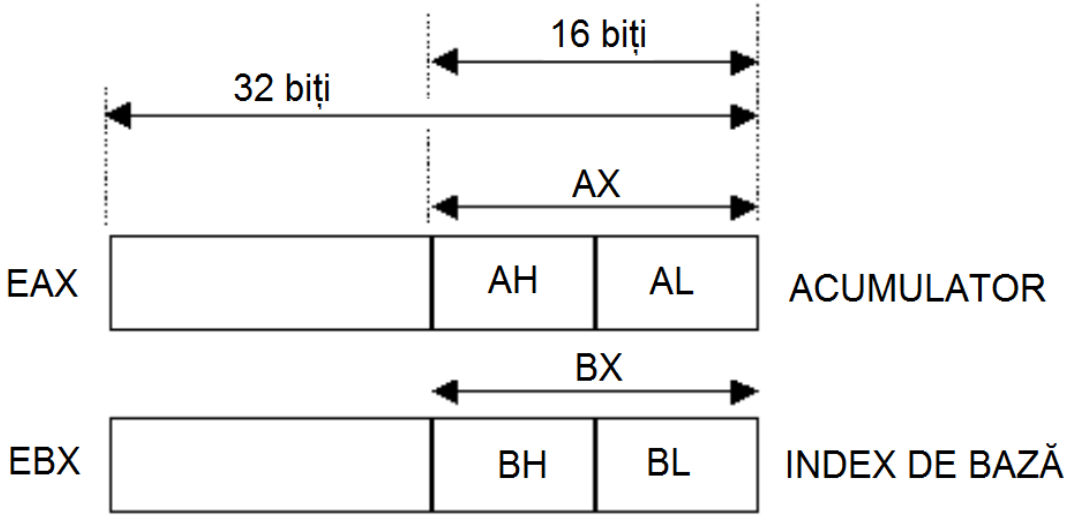
Din 2002 au aparut seriile (enumerare selectivă):

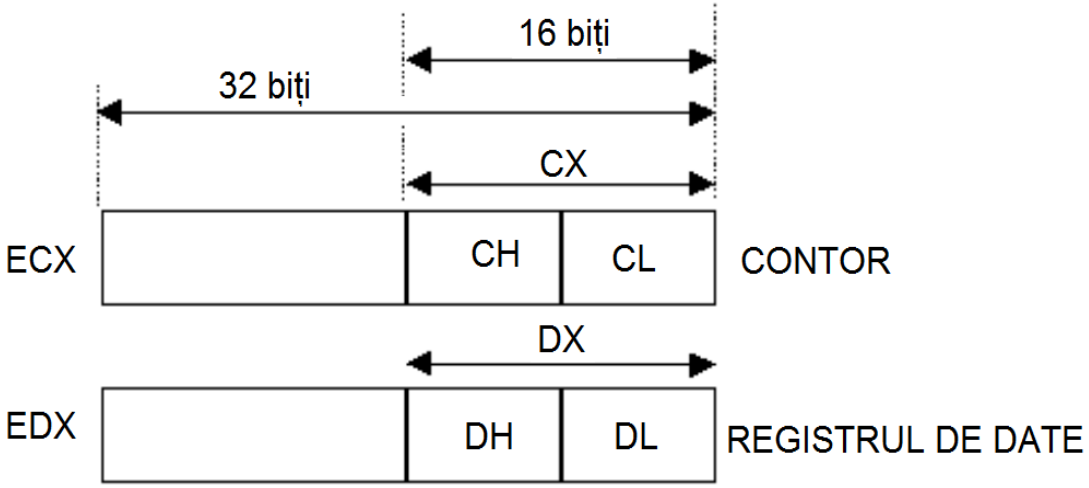
- Socket 604 (cu 604 pini, Intel Xeon),
- Socket 754 (AMD Athlon 64),
- Socket 940 (AMD Optheron Athlon 64 FX – 1 GHz),
- Socket F (2006 – 1207 pini,
- AMD Athlon 64 FX – 2 GHz, Dual CPU mode),
- Socket AM3 (2009, 941/940 pini, AMD Phenom II, 3.2GHz),
- Socket H2 (2011, 1155 pini, Intel Sandy Bridge, 5.7 GT/s – GigaTransfer per Second),
- Socket R,.. ,
- Socket AM1 (2014, 721 pini, AMD Athlen, AMD Sempron)
- Socket AM4 (2017,1331 pini, AMD Ryzen, 4 GHz ?).



Socket AM4 Ryzen

Regiștrii





Regiștrii de uz general: Acumulatorul și Indexul de bază

Regiștrii de uz general: Contorul și Registrul de date

Toţi regiştrii procesorului 8086 sunt pe 16 biţi.

O serie de regiştrii (AX, BX, CX, DX) sunt disponibili şi la nivel de octet.
Părţile mai semnificative sun **AH, BH, CH şi DH**, iar cele mai puţin semnificative **AL, BL, CL şi DL**.

Regiștrii

Denumirile regiştrilor sunt:
- *AX* -registru acumulator
- *BX* -registru de bază general
- *CX* -registru contor
- *DX* -registru de date

- *BP* -registru de bază pentru stivă (base pointer)
- *SP* -registru indicator de stivă (stack pointer)

- *SI* -registru index sursă
- *DI* -registru index destinaţie

- *IP* -contorul program.

Registrul **IP** (instruction pointer) este contorul program.
Registrul **FLAGS** cuprinde flagurile procesorului şi ale bistabililor de condiţie.

Denumirile regiştrilor de segment sunt:
- CS -registrul de segment de cod (code segment)
- DS -registru de segment de date (data segment)
- SS -registru de segment de stivă (stack segment)
- ES -registru de segment suplimentar de date (extra segment).

OBSERVAŢII

1. Perechea de regiştrii (***CS:IP***) va indica adresa următoarei instrucţiuni

2. Perechea (***SS:SP***) indică adresa vârfului stivei

3. Regiştrii ***DS şi ES*** sunt folosiţi la accesarea datelor.

Moduri de adresare

Modul de adresare specifică metoda prin care se calculează adresa fizică a operandului aflat în memorie.
Adresa fizică (AF) se calculează utilizând:

- (AS) adresa de segment adică adresa de început a segmentului unde se află operandul;
- (AE) adresa efectivă adică offset-ul operandului din cadrul segmentului.

CLASIFICARE:

1. Adresarea imediată
2. Adresare directă
3. Adresare indirectă (prin regiştrii)
4. Adresarea indexată (sau bazată)

Moduri de adresare

1. Adresarea imediată

Operand = Valoare

Operandul apare explicit în instrucţiune:

```
mov  ax, 3    ; pune în AX valoarea 3
add  bx, 4  ; adună la BX valoarea 4
```

Sau ? – testați dacă funcționează, ca exercițiu!

```
 ADD 5  ;adună 5 la conţinutul
acumulatorului (implicit)
```

Moduri de adresare

2. Adresare directă

EA = A

Unde A este adresa celulei de memorie.

Adresa efectivă (EA) a operandului este furnizată printr-un offset. Implicit se foloseşte registrul de segment DS, în cazul în care nu se face o explicitare a registrului de segment.

```
.data
VAL dw 4
.code
……————————————
mov  bx, VAL   ;pune în registrul bx valoarea 4
add  cx, [100] ;adună la registrul cx ceea ce se află în memorie,
               ; în segmentul de date la offset-ul 100.
```

VAL se va înlocui cu deplasamentul din segmentul de date, la asamblare. Deplasamentul este dat explicit cu valoarea 100.

Sau ? – testați dacă funcționează, ca exercițiu!

```
ADD A            ; adună conţinutul locaţiei A la acumulator
```

Moduri de adresare

3. Adresare indirectă (prin regiştrii)

EA = [R]

Operatorul va avea adresa efectivă (EA) dată de conţinutul următorilor regiştrii:

- BX-registru de bază general,
- SI-registru index sursă şi
- DI-registru index destinaţie.

Registrul de segment de date implicit este DS.

```
……................................
mov ax, [bx]    ; pune în AX conţinutul locaţiei de memorie
                ;de la adresa dată de BX
mov [di], cx    ; memorează conţinutul lui CX
                ;la adresa dată de registrul DI
add byte ptr [si], 7  ; adună valoarea 7 la octetul
                      ;aflat la adresa dată de SI
```

Instrucţiunea ADD foloseşte operatorul "byte ptr" care este absolut necesar, altfel nu s-ar cunoaste tipul/lungimea operandului (octet, cuvânt) la care se adună 7.

Moduri de adresare

4. **Adresarea indexată (sau bazată)**

EA = X + [R]

Operandul din memorie va avea adresa efectivă **(EA)** obţinută prin adunarea unui offset constant **(X)** de 8 sau 16 biţi la unul din regiştrii de bază (**BP sau BX**)
sau la unul din regiştrii index (**SI sau DI**)
ce conţin adresa locaţiei de memorie.

Pentru **BX şi SI** registrul de segment implicit este **DS**.
Pentru **BP** registrul de segment implicit este **SS**.

Tipuri de date

Tip date	Directivă
BYTE	DB
DOUBLE-WORD	DD
QUAD-WORD	DQ
TEN-BYTES	DT

1. **BYTE** (1 octet = 8 biţi)

Definirea datelor de acest tip se face cu directiva ***DB*** *(Define Byte).*
Reprezentarea poate fi făcută în memoria internă sau într-un registru de 8 biţi al procesorului.

BYTE-ul poate fi interpretat ca:

- caracter ASCII;
- întreg pe 8 biţi cu sau fără semn.

Tipuri de date

Tip date	Directivă
BYTE	DB
DOUBLE-WORD	DD
QUAD-WORD	DQ
TEN-BYTES	DT

2. WORD (2 octeţi = 16 biţi)

Definirea datelor de acest tip se face cu directiva **DW** *(Define Word).*
Reprezentarea poate fi făcută în memoria internă sau într-un registru de 16 biţi al procesorului.

WORD poate fi interpretat ca:

- secvenţă de două caractere ASCII;
- întreg pe 16 biţi cu sau fără semn;
- adresă memorie de 16 biţi.

Octetul mai puţin semnificativ este memorat la adrese mici.

De exemplu dacă presupunem întregul 35794H la adresa 1000H, atunci:

- octetul 79H se va găsi la adresa 1000H,
- octetul 35H la adresa 1001H.

Secvenţele de două caractere ASCII se memorează similar.

Tipuri de date

Tip date	Directivă
BYTE	DB
DOUBLE-WORD	DD
QUAD-WORD	DQ
TEN-BYTES	DT

3. DOUBLE-WORD (4 octeţi = 32 biţi)

Definirea datelor de acest tip se face cu directiva **DD** *(Define Double-Word).*

Reprezentarea poate fi făcută în memoria internă sau într-o pereche de regiştrii de 16 biţi sau într-un registru de 32 de biţi (la procesoarele de 32 de biţi).

DOUBLE-WORD poate fi interpretat ca:

- întreg pe 32 de biţi cu sau fărăsemn;
- număr real în simplă precizie;
- adresăde memorie de 32 de biţi.

Octeţii mai puţin semnificati se memorează la adrese mici.

În cazul adreselor pe 32 de biţi, adresa de segment este memorată la adrese mari, iar la adrese mici offset-ul (deplasamentul).

Tipuri de date

Tip date	Directivă
BYTE	DB
DOUBLE-WORD	DD
QUAD-WORD	DQ
TEN-BYTES	DT

4. QUAD-WORD (8 octeţi)

Definirea datelor de acest tip se face cu directiva **DQ** *(Define Quad-Word).*

Reprezentarea poate fi făcută în memoria internă sau într-o pereche de regiştrii de 32 de biţi (numai la procesoarele de 32 de biţi).

QUAD-WORD poate fi interpretat ca:

- întreg pe 64 de biţi cu sau fărăsemn;
- număr real în dublă precizie;

5. TEN-BYTES (10 octeţi)

Definirea datelor de acest tip se face cu directiva **DQ** *(Define Quad-Word).*

Reprezentarea poate fi făcută în memoria internă sau într-unul din regiştrii coprocesoarelor matematice.

TEN-BYTES poate fi interpretat ca:

- număr întreg reprezentat ca secvenţă de cifre BCD împachetate, cu semn memorat explicit;
- număr real în precizie extinsă.

Tipuri de date

Exercițiu

În vederea comparării unor instrucţiuni din C cu echivalentul lor din Assembler prezentăm o secvenţă simplă de instrucţiuni C:

```
....
        int x,y;
        x=56;
        y=x+3;
        y=x+1;
        y=x++;
        y=++x;
....
        puts („Bun venit !!”);
```

Veţi transpune aceste instrucţiuni în Assembler, ca temă pentru acasă.